

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA GRAPH RAG EN APLICACIONES CULTURALES



INTELIGENCIA ARTIFICIAL EN APLICACIONES CULTURALES
CURSO 2025-2026

AUTORES

SANDRA CONDE GONZÁLEZ
PABLO FOLGUEIRA GALÁN

RESPOSITORIO DE GITHUB:

[HTTPS://GITHUB.COM/PFOLGUEIRA/GRAPHRAG-IAC](https://github.com/pfolgueira/graphrag-iac)

MÁSTER EN INTELIGENCIA ARTIFICIAL
FACULTAD DE INFORMÁTICA
UNIVERSIDAD COMPLUTENSE DE MADRID

Índice general

1	Hito 1: Selección de Dominio y Modelado de Datos	2
1.1	Dominio y preguntas	2
1.1.1	Consultas semánticas	2
1.1.2	Consultas estructuradas	2
1.1.3	Consultas híbridas	3
1.2	Fuentes de ingesta	3
1.3	Modelo de datos Neo4j	4
1.3.1	Etiquetas, Propiedades y Relaciones	4
1.3.2	Justificación del modelo de datos	7
2	Hito 2: Construcción Automática del Grafo de Conocimiento	10
2.1	Estrategias de <i>chunking</i>	10
2.2	Extracción de entidades y relaciones	11
2.2.1	Extracción multi-paso	11
2.3	Resolución de Entidades (<i>Entity Resolution</i>)	14
2.3.1	Resolución de entidades mediante <i>enums</i>	14
2.3.2	Resolución de entidades mediante descripciones	14
2.3.3	Resolución dinámica de especies (<i>Species</i>) mediante LLM	14
2.4	Preprocesamiento para RAG	17
3	Hito 3: Implementación y Evaluación de <i>Retrievers</i>	21
3.1	Retriever Text2Cypher	21
3.2	Retriever RAG Vectorial	23
3.2.1	Recuperación vectorial (<i>Vector Retriever</i>)	24
3.2.2	Recuperación por palabras clave (<i>Full-Text Retriever</i>)	25
3.2.3	Recuperador híbrido y Reranking (<i>Hybrid Retriever</i>)	25
3.3	Retriever <i>Queries</i> Manuales	26
4	Hito 4: Orquestación Agéntica y Evaluación Final	28
4.1	Sistema Agéntico	28
4.1.1	El Agente Enrutador (<i>Router</i>)	28
4.1.2	Agente Razonador Multipaso (<i>Multi-hop Reasoner</i>)	30
4.1.3	Generación de la Respuesta (<i>Synthesizer Agent</i>)	32
4.1.4	Evaluación Continua: Agente Crítico y Ciclo de Refinamiento	33
4.2	Evaluación del Sistema	33
4.2.1	Diseño del <i>Benchmark</i>	33
4.2.2	Métricas de Evaluación	34

1. Hito 1: Selección de Dominio y Modelado de Datos

1.1. Dominio y preguntas

Con el objetivo de desarrollar un sistema de Graph RAG capaz de responder tanto a preguntas de carácter semántico como a preguntas estructuradas, hemos decidido centrarnos en el dominio de la zoología y el mundo animal. Tras haber analizado distintos dominios, consideramos que este encajaba con el propósito del proyecto, ya que combina información de los dos tipos que estamos buscando. Por un lado, dispone de información no estructurada, como descripciones del aspecto, comportamiento, reproducción o alimentación de las distintas especies y, por otro lado, también cuenta con datos estructurados, como la taxonomía de las distintas familias de animales, las localizaciones geográficas de los hábitats y parámetros biométricos como el tamaño, el peso o la esperanza de vida de un animal concreto.

Con esta información disponible, la idea es construir un sistema Graph RAG que pueda responder a distintos tipos de preguntas, como las que se muestran a continuación:

1.1.1. Consultas semánticas

- Describe cómo es la migración del halcón peregrino.
- ¿Cuál es la apariencia de un tiburón blanco?
- ¿Cómo cazan los leones a sus presas?
- ¿Podrías decirme algunos datos curiosos sobre las anacondas?
- ¿Cómo es la hibernación de los osos pardos? ¿Por qué lo hacen?
- Describe el comportamiento social de las orcas y cómo son capaces de comunicarse entre ellas.

1.1.2. Consultas estructuradas

- ¿Qué tamaño tiene un delfín?
- ¿Cuál es el hábitat natural de los canguros?
- ¿Cuáles son las presas de los leones?
- ¿Cuáles son los depredadores de las gacelas?
- ¿Cuánto dura el periodo de incubación de un pingüino emperador?
- ¿Cuántas especies de la familia *Felidae* se encuentran en el estado de conservación “En Peligro”?
- ¿Cuántos animales acuáticos hay en la base de datos? ¿Cuántos de ellos son peces?
- ¿Qué tipos de alimentación se repiten con más frecuencia entre los felinos?

1.1.3. Consultas híbridas

- De las especies que habitan en la Sabana, ¿cuáles son sus diferentes estrategias de supervivencia ante la sequía?
- Compara las dietas de los felinos que habitan en África frente a los de Asia.
- De los animales ovíparos, ¿en qué especies el macho se encarga de cuidar a la cría?
- ¿Cómo cuidan a sus crías las hembras de especies que habitan en el Polo Norte?
- De los animales con una esperanza de vida mayor a 25 años, ¿cuáles son sus principales estrategias de defensa?
- Compara las capacidades cognitivas más desarrolladas de los animales que pertenecen a la familia de los simios.
- Analiza si existe un patrón común en las estrategias de defensa de los animales que habitan en la selva frente a los de zonas desérticas.

1.2. Fuentes de ingesta

Para el desarrollo de nuestro sistema hemos analizado distintas fuentes de datos dentro de nuestro área de aplicación, teniendo en cuenta la limitación de tiempo de procesamiento que conlleva la construcción automática del grafo y la extracción de las distintas entidades con sus características mediante LLMs. Esto nos llevó a centrarnos en enciclopedias y sitios web con artículos especializados, que creemos que pueden aportarnos toda la información necesaria para el desarrollo de nuestro sistema. Otro aspecto a tener en cuenta es que, al tratarse de un dominio tan extenso donde existen una gran cantidad de animales y subespecies muy específicas, hemos decidido comenzar con un subconjunto representativo de los animales más populares y, posteriormente, ampliarlo si fuese posible.

- Wikipedia: https://en.wikipedia.org/wiki/List_of_animal_names

La fuente de datos principal es Wikipedia, debido a que contiene información detallada sobre muchos aspectos de los animales como sus características, sus diferentes comportamientos o los lugares en los que habitan. En el enlace que hay al comienzo de este apartado aparece una tabla con el título “*Terms by species or taxon*”, en la que aparecen 183 animales de distintos tipos y especies, por lo que puede ser un buen punto de partida para comenzar a desarrollar nuestro sistema.

Para acceder a la información crearemos un pequeño *scraper* que obtenga los nombres de los animales de la tabla “*Terms by species or taxon*”. Una vez obtenidos, utilizaremos la librería `wikipedia-api` (disponible en Python) para realizar consultas a la API de Wikipedia, que nos permite hacer una petición de forma sencilla para obtener todo el texto de una página directamente. De esta forma accederemos a la información de cada uno de los animales obtenidos en el paso anterior y, posteriormente, procesaremos el texto para almacenarlo en formato *markdown* y poder enviárselo directamente al LLM en fases posteriores del desarrollo.

- A-Z Animals: <https://a-z-animals.com/animals/>

Esta fuente de datos dispone de una lista muy extensa de especies de animales ordenada alfabéticamente. En este caso, los textos no son tan largos como en la fuente anterior pero tienen todos los apartados muy organizados y aportan información adicional como aspectos y características interesantes de un animal.

Consideramos que puede ser una fuente útil para añadir nuevos animales, además de los obtenidos de la fuente anterior, y para enriquecer la información del sistema añadiendo datos y características nuevas. La página web de cada animal dispone de un apartado llamado “*article*”

donde aparece toda su información de forma textual y, para poder obtenerla, será necesario hacer *web scraping*.

- Animalia: <https://animalia.bio/all>

Por último, la web de Animalia también contiene una lista con muchos animales, que se puede ordenar por popularidad y que podríamos utilizar para ampliar el número de animales de nuestro sistema y aportar datos más específicos, curiosidades o aspectos divertidos para niños. Igual que en el caso anterior tendríamos que hacer *web scraping* para acceder a cada uno de los apartados que contienen la información del animal y componer el texto completo antes de pasar a la extracción de entidades.

1.3. Modelo de datos Neo4j

1.3.1. Etiquetas, Propiedades y Relaciones

Species

Propiedades	Descripciones
name	Nombre común de la especie en lenguaje coloquial.
weight_min_kg	Peso mínimo registrado de un ejemplar adulto en kilogramos.
weight_max_kg	Peso máximo registrado de un ejemplar adulto en kilogramos.
length_min_m	Longitud mínima registrada del cuerpo en metros.
length_max_m	Longitud máxima registrada del cuerpo en metros.
height_min_m	Altura mínima registrada hasta el punto más alto del animal en metros.
height_max_m	Altura máxima registrada hasta el punto más alto del animal en metros.
top_speed_kmh	Velocidad máxima que la especie puede alcanzar.
lifespan_years	Esperanza de vida de la especie.
gestation_period_days	Duración promedio del desarrollo del embrión hasta el nacimiento en días.
maturity_age_years	Edad estimada en años en la que la especie alcanza la madurez sexual o independencia.

Relaciones	Etiquetas Destino	Descripciones
MEMBER_OF_FAMILY	Family	Indica la familia taxonómica a la que pertenece la especie dentro de la clasificación biológica.
BELONGS_TO_CLASS	AnimalClass	Especifica la clase biológica de la especie.
HAS_SKELETAL_STRUCTURE	SkeletalStructure	Define el tipo de estructura esquelética de la especie.
REPRODUCES_VIA	ReproductionMethod	Indica el método de reproducción de la especie.
LIVES_IN_ENVIRONMENT	EnvironmentType	Describe el medio físico donde vive la especie.
INHABITS	Habitat	Relaciona la especie con el tipo de hábitat o bioma donde se encuentra habitualmente.
FOUND_IN	Location	Señala las regiones geográficas donde la especie está presente de forma natural.
MIGRATES_TO	Location	Indica las regiones hacia las que la especie se desplaza durante procesos migratorios.
<i>Propiedad [season]: Indica la estación del año en que se realiza la migración.</i>		
HAS_ACTIVITY_CYCLE	ActivityCycle	Describe el periodo del día en el que la especie es principalmente activa.
ORGANIZED_IN	SocialStructure	Representa la forma de organización social de la especie.
HAS_DIET_TYPE	DietType	Define el tipo de dieta alimentaria de la especie.
PREYS_ON	Species	Indica que la especie caza y consume a otra especie como parte de su dieta.
FEEDS_ON	FoodSource	Representa el consumo de fuentes de alimento no animales.
HAS_CONSERVATION_STATUS	ConservationStatus	Indica el estado de conservación de la especie según su riesgo de extinción.

Family

Propiedad	Descripción
type	Categoría de la clasificación taxonómica que agrupa varios géneros de especies con características evolutivas y morfológicas similares: Hominidae, Felidae, Canidae, Equidae, Elephantidae, Bovidae, etc.

AnimalClass

Propiedad	Descripción
type	Categoría biológica principal: Mammal, Reptile, Bird, Fish, Amphibian.

SkeletalStructure

Propiedad	Descripción
type	Clasificación anatómica: Vertebrate, Invertebrate.

ReproductionMethod

Propiedad	Descripción
type	Mecanismo de desarrollo embrionario: Oviparous, Viviparous, Ovoviviparous.

EnvironmentType

Propiedad	Descripción
type	Medio físico principal donde la especie desarrolla su ciclo vital: Aquatic, Terrestrial, Aerial.

Habitat

Propiedad	Descripción
type	Tipo de entorno natural o ecosistema en el que vive una especie y al que está adaptada: Forest, Desert, Grassland, Savanna, Ocean, Freshwater, Mountain, etc.

Location

Propiedad	Descripción
type	Región geográfica o área del mundo donde se encuentra una especie animal de forma natural: Africa, South America, Europe, Amazon Rainforest, Arctic, Pacific Ocean, Atlantic Ocean, etc.

ActivityCycle

Propiedad	Descripción
type	Patrón temporal de actividad biológica: Nocturnal, Diurnal, Crepuscular.

SocialStructure

Propiedad	Descripción
type	Patrón de organización social de una especie: Solitary, Pair-living, Family group, Herd (grupo sin jerarquía estricta. Ej: ciervos), Pack (grupo cooperativo con jerarquía. Ej: lobos), Colony (agrupación para reproducción o refugio. Ej: pingüinos), Eusocial (organización altamente estructurada. Ej: abejas, hormigas).

DietType

Propiedad	Descripción
type	Clasificación según la dieta alimentaria: Carnivore, Herbivore, Omnivore.

FoodSource

Propiedad	Descripción
type	Tipo de recurso vegetal del que se alimenta una especie herbívora: Grass, Leaves, Fruits, Seeds, Bark, Nectar, Aquatic Plants.

ConservationStatus

Propiedad	Descripción
type	Nivel de riesgo de extinción según IUCN: EX (Extinct), EW (Extinct in the Wild), CR (Critically Endangered), EN (Endangered), VU (Vulnerable), NT (Near Threatened), LC (Least Concern), DD (Data Deficient) y NE (Not Evaluated).

1.3.2. Justificación del modelo de datos

El diseño del modelo de datos se deriva directamente de las fuentes de ingesta mencionadas en la sección 1.2 (Wikipedia, A-Z Animals y Animalia). Estas fuentes proporcionan datos estructurados y textuales sobre taxonomía, características físicas, hábitos, alimentación, comportamiento y conservación de las especies, lo que ha permitido construir un modelo que soporte consultas diversas sobre animales como las que se presentan en la sección 1.1.

Las entidades principales de nuestro modelo de datos están representadas por la etiqueta **Species**, que se refiere a una especie de animal concreta y tiene como propiedades los atributos cuantitativos propios de esa especie tales como su tamaño, peso o esperanza de vida. Estos datos pueden obtenerse directamente de la sección de características y dimensiones de cada animal llamada “Description” en Wikipedia, así como de apartados sobre sus patrones de desarrollo y reproducción como “Reproduction and life cycle” o “Longevity”, permitiendo dar respuesta a preguntas estructuradas sobre el grafo como “¿Cuál es la altura máxima de una jirafa?”, “¿Qué velocidad puede alcanzar un guepardo?” o servir como punto de partida para consultas híbridas como “De los animales con una esperanza de vida mayor a 25 años, ¿cuáles son sus principales estrategias de defensa?”.

Por su parte, **Species** se asocia con otras etiquetas por medio de relaciones que pueden definirse procesando diferentes secciones en las fuentes de ingesta:

- **MEMBER_OF_FAMILY**: Esta relación nos permite relacionar una especie con la familia a la que pertenece (etiqueta **Family**) para poder responder a preguntas tanto estructuradas como híbridas: “¿Cuántas especies de la familia Felidae se encuentran en el estado de conservación “En Peligro”?” o “Compara las capacidades cognitivas más desarrolladas de los animales que pertenecen a la familia de los simios.”.

La etiqueta **Family** representa una categoría taxonómica que agrupa varias especies distintas que tienen características similares, por ejemplo, Hominidae, Felidae, Equidae, etc. Hay casos de textos de la API de Wikipedia donde la familia no aparece explícitamente en el texto y el LLM

encargado de extraer las entidades no tendrá forma de reconocerlas, por lo que hemos decidido que podemos incluirla a partir de la información de A-Z Animals haciendo *web scraping*.

- **BELONGS_TO_CLASS**, **HAS_SKELETAL_STRUCTURE** y **REPRODUCES_VIA**: Estas relaciones pueden establecerse a partir de las secciones que describen la clasificación biológica, anatómica o el mecanismo de reproducción de la especie. No obstante, hay artículos completos de animales que pueden no contener esta información y el LLM no podrá reconocerlos correctamente. Por ello, si fuese necesario, podríamos realizar *web scraping* para obtener los parámetros estructurados de A-Z Animals o Animalia y así poder incluirlos.

La relación “**BELONGS_TO_CLASS**” permite asociar un animal con la clase a la que pertenece (mamífero, reptil, ave, anfibio, ave o pez) gracias a la etiqueta **AnimalClass**. Por su parte, la etiqueta **SkeletalStructure** representa si un animal es vertebrado o invertebrado y, **ReproductionMethod** si un animal es ovíparo, vivíparo u ovovivíparo. Estas relaciones nos permiten responder preguntas como “De los animales ovíparos, ¿en qué especies el macho se encarga de cuidar a la cría?” o “¿Qué animales mamíferos/vertebrados hay en la base de datos?”.

- **LIVES_IN_ENVIRONMENT**, **INHABITS**, **FOUND_IN** y **MIGRATES_TO**: Estas cuatro relaciones se refieren al tipo de entorno en el que habitan las especies, por lo que apartados como “Distribution and habitat”, “Migration” o “Migratory Patterns” de Wikipedia permiten crearlas en el grafo.

En este caso hemos hecho una distinción entre el entorno en el que habitan (relación “**LIVES_IN_ENVIRONMENT**”), representado con la etiqueta **EnvironmentType**, y el hábitat de la especie (relación “**INHABITS**”), representada con **Habitat**, con el objetivo de distinguir si una especie es terrestre, acuática o voladora del hábitat concreto en el que habitan (por ejemplo la selva, la sabana o el desierto). Esto nos permite responder a preguntas como “¿Cuál es el hábitat natural de los canguros?” o “¿Cuántos animales acuáticos hay en la base de datos?”.

Por otro lado, las relaciones “**FOUND_IN**” y “**MIGRATES_TO**” unen una especie con etiquetas **Location**. La primera representa el lugar exacto donde puede encontrarse una especie, mientras que la segunda es útil para reflejar hacia dónde migra una especie concreta y en qué época del año lo hace, gracias a la propiedad de la relación **season**. Estas relaciones nos permiten dar respuesta a preguntas como “¿Qué animales migran a Europa? ¿En qué época del año?” o “¿En qué lugar pueden encontrarse los elefantes?”.

- **HAS_ACTIVITY_CYCLE** y **ORGANIZED_IN**: La información para estas relaciones puede obtenerse a partir de secciones que describen el comportamiento de la especie como “Behaviour” o “Social Behaviour” en el caso de Wikipedia. La relación “**HAS_ACTIVITY_CYCLE**” pretende representar el patrón de actividad biológica (nocturno, diurno o crepuscular) de una especie uniéndola con la etiqueta **ActivityCycle** y, por su parte, “**ORGANIZED_IN**” se refiere a su organización social, es decir, si vive solo (*Solitary*), en un grupo sin jerarquía estricta (*Herd*) o con jerarquía (*Pack*) entre otros. Estos elementos se representan con la etiqueta **SocialStructure**.

Tener recogida esta información permite responder a preguntas del tipo: “¿Qué animales desarrollan su actividad principalmente por la noche?” o “¿Qué especies desarrollan su vida de forma solitaria?”.

- **HAS_DIET_TYPE**, **PREYS_ON**, **FEEDS_ON**: Estas relaciones se refieren a la dieta alimentaria de la especie, por lo que el apartado “Hunting and diet” para los depredadores o “Behaviour and ecology” para los no depredadores pueden aportarnos esta información. Las preguntas que pueden ser respondidas son “¿Cuáles son las presas de los leones?” o “¿Cuáles son los depredadores de las gacelas?”.

La relación “**HAS_DIET_TYPE**” permite clasificar y relacionar distintas especies según su tipo de alimentación (carnívora, herbívora u omnívora), que representamos con la etiqueta **DietType**. Por su parte, “**PREYS_ON**” permite representar la relación presa-depredador directamente con otro nodo de etiqueta **Species** y, por último, “**FEEDS_ON**” es útil para asociar especies no

depredadoras con aquellos recursos que les sirven de alimento (etiqueta ***FoodSource***) y que aparecen mencionados a lo largo de los apartados del párrafo anterior.

- **HAS_CONSERVATION_STATUS**: Esta última relación puede obtenerse de apartados como “Conservation” de Wikipedia, aunque algunas especies no disponen de dicha información, por lo que podría hacerse *web scraping* de otras fuentes de datos como A-Z Animals. Para representar el estado de conservación hemos decidido crear la etiqueta ***ConservationStatus***, que representa el nivel de riesgo de extinción según la IUCN. De esta forma, podemos establecer una conexión entre distintas especies en base a este nivel de amenaza para responder a preguntas como “¿Cuántas especies de animales se encuentran en el estado de conservación “En Peligro”?” o “¿Qué especies se encuentran en estado “Vulnerable”?”.

2. Hito 2: Construcción Automática del Grafo de Conocimiento

2.1. Estrategias de *chunking*

Nuestra idea inicial era utilizar un corpus basado en textos de la Wikipedia. Sin embargo, tras hacer algunas pruebas, nos dimos cuenta de que contenían mucha información ruidosa que afectaba al contenido del grafo o que no se alineaba con lo que pretendíamos representar. Por ejemplo, había muchas referencias a especies extintas que confundían bastante a los modelos de lenguaje o, en lugar de englobar comportamientos de una especie a nivel general, los fragmentos hacían referencia a subespecies, lo cual no era nuestro objetivo inicial. Por ello, decidimos realizar *web scraping* del apartado *Article* de la fuente “A-Z Animals”, estructurandola en formato *markdown* para facilitar el procesamiento necesario para implementar la estrategia de *chunking*. En esta jerarquía, el título principal (#) se corresponde con el animal o especie actual, seguido de las distintas secciones y subsecciones que componen su artículo.

Para procesar estos textos de la manera más adecuada, seguimos una estrategia de segmentación o *chunking* dividida en dos fases:

- **Segmentación semántica por secciones:** En primer lugar, los documentos se dividieron a partir de los encabezados *markdown*, de forma que cada bloque resultante tratase una temática específica del animal, como puede ser su hábitat, dieta o comportamiento, separando esta información de manera coherente antes de realizar subdivisiones más detalladas.
- **Segmentación recursiva por caracteres:** Dentro de cada sección, aplicamos el método *Recursive Character Splitting*. Este enfoque jerárquico intenta realizar los cortes priorizando la estructura lógica del texto: primero divide por párrafos (`\n\n`), luego por oraciones (`\n` o `.`) y, finalmente, por palabras (`' '`), manteniendo el contexto semántico en la medida de lo posible.

Tras evaluar distintas configuraciones, establecimos un tamaño máximo de fragmento (*chunk size*) de 800 caracteres. De este modo, si un párrafo supera este límite, el algoritmo lo subdivide recursivamente en unidades menores. Además, para los casos en los que se alcance el nivel de división más granular y se siga superando el tamaño máximo de 800 caracteres, configuramos un solapamiento (*overlap*) de 80 caracteres, asegurando que el sistema no pierda información en los extremos de los fragmentos. La implementación de esta lógica la realizamos utilizando la configuración de la figura 2.1

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=800,  
    chunk_overlap=80,  
    separators=["\n\n", "\n", r"(?<=\. )", " "],  
    is_separator_regex=True,  
    length_function=len,  
)
```

Figura 2.1: Implementación de la lógica de *chunking*.

Por último, para evitar que se pierda el contexto entre diferentes *chunks* y proporcionar la información necesaria al LLM, decidimos incluir al comienzo de cada uno de ellos el nombre del animal que se

estaba procesando, así como la jerarquía de la sección y subsección a la que pertenece dicho fragmento. El resultado final de un *chunk* procesado siguiendo esta estrategia se muestra en la figura 2.2.

Animal: Lion
Section: Evolution and Origins
Lions among most members of the cat family are thought to be descended from a common ancestor known as the Proailurus Lemanensis, which translates to “first cat”. This ancient creature was a cat-like creature that walked the earth nearly 25 million years ago.
Fossil evidence suggests that the earliest lion-like cat appeared in Laetoli in Tanzania in East Africa during the Late Pliocene (5.0–1.8 million years ago).
Additionally, genetic studies suggest that the lion evolved in eastern and southern Africa.

Figura 2.2: Texto de ejemplo correspondiente a un chunk procesado siguiendo la estrategia de *chunking*.

2.2. Extracción de entidades y relaciones

Una vez definida e implementada la estrategia de *chunking*, pasamos a evaluar el comportamiento de distintos LLMs para la tarea de extracción de entidades y relaciones. Durante las fases iniciales del proyecto, experimentamos con modelos de ejecución local haciendo uso de Ollama, concretamente Qwen3 (4B) y Llama 3.1 (8B).

Por un lado, el modelo Qwen permitía una extracción muy precisa gracias a su gran capacidad de razonamiento, pero las limitaciones del hardware provocaban tiempos de espera entre respuesta excesivos, lo que haría muy complicada la extracción para el corpus completo. Por otro lado, Llama 3.1 respondía de forma más directa al omitir algunos pasos de razonamiento, pero esto resultó en un aumento del número de alucinaciones del modelo. Además, el tiempo de procesamiento seguía siendo mucho mayor del esperado para poder realizar las pruebas necesarias y completar el grafo con toda la información recogida, por lo que la idea de ejecutar estos modelos en local por medio de Ollama terminamos descartándola.

Debido a estas limitaciones de hardware y rendimiento, optamos por utilizar el modelo Gemini 2.5 Flash-Lite, al que accedimos a través de la API de Google AI Studio, y que permite un procesamiento muy eficiente y considerablemente rápido de toda la información extraída en la fase de *chunking*, al mismo tiempo que ofrece un coste económico relativamente bajo. No obstante, durante las pruebas iniciales de extracción se encontraron ciertas limitaciones relacionadas con la falta de razonamiento profundo del modelo en comparación con otros modelos como Gemini 2.5 Flash o Gemini 2.5 Pro, cuya elección habría resultado en un coste por *token* superior a la hora de procesar el corpus completo. Los dos problemas principales que se encontraron utilizando el modelo Gemini 2.5 Flash-Lite fueron: la omisión de entidades relevantes y la generación de alucinaciones, como ignorar negaciones explícitas en el texto dando lugar a relaciones incorrectas (por ejemplo, el modelo extrajo la relación *Lion* ->INHABITS ->Desert de un fragmento que indicaba expresamente que los leones *no* habitan en desiertos). Para mitigar estos problemas y deficiencias del modelo Gemini 2.5 Flash-Lite, se definió una arquitectura de extracción multi-paso.

2.2.1. Extracción multi-paso

Para comenzar el proceso de extracción de entidades y relaciones de los *chunks*, definimos los *prompts* que se muestran en la figura 2.3, asignándole al LLM el rol de ser un experto en construir grafos de conocimiento sobre zoología y diciéndole que sus objetivos eran extraer entidades y relaciones a partir del texto de entrada. Además, le insistimos en que no utilice conocimiento externo y se base

únicamente en la información del chunk que está recibiendo, así como que intente ajustarse lo máximo posible al esquema *Pydantic* que habíamos definido en el modelo de datos de la sección 1.3.

System Prompt: “You are an expert zoology Knowledge Graph extractor. Your goal is to extract animal species, their biometrics, their classifications, and specific relationships from the text provided.

CRITICAL RULES (ZERO TOLERANCE FOR HALLUCINATIONS):

1. STRICT GROUNDING: You must base your extraction on the source text. DO NOT use pre-trained knowledge, common sense, or assumptions.
2. SCHEMA & ENUM COMPLIANCE: Strictly adhere to the provided schema descriptions and ENUM constraints.
3. ENTITY CONSISTENCY: Ensure the source and target of each relationship exactly match the extracted entities.

User Prompt: “Extract zoological entities and their relationships from the following text: {Chunk text}”

Figura 2.3: *System prompt* y *user prompt* diseñados para la primera fase de la extracción de entidades y relaciones.

Con el objetivo de limitar las alucinaciones lo máximo posible, dentro de nuestro modelo de datos definimos diferentes *enums* para algunas entidades como, por ejemplo, el estado de conservación, que debería tomar únicamente los valores de la figura 2.4. Por otro lado, intentamos proporcionarle descripciones claras que le permitan comprender exactamente a qué se refiere cada relación e identificar las entidades que debe extraer. Uno de estos casos son las relaciones *PREYS_ON* y *FEEDS_ON*, en las que tratamos de indicarle claramente que la primera sólo debe utilizarse para relacionar un animal carnívoro (u omnívoro) con otra especie de animal a la que depreda y, en la segunda, que representa a un animal herbívoro (u omnívoro) que se alimenta de cualquier otro tipo de recurso no animal (figura 2.5).

```
class ConservationStatusEnum(str, Enum):
    EX = "EX (Extinct)"
    EW = "EW (Extinct in the Wild)"
    CR = "CR (Critically Endangered)"
    EN = "EN (Endangered)"
    VU = "VU (Vulnerable)"
    NT = "NT (Near Threatened)"
    LC = "LC (Least Concern)"
    DD = "DD (Data Deficient)"
    NE = "NE (Not Evaluated)"
```

Figura 2.4: Ejemplo de *enum* definido para las entidades *Conservation Status*.

Con toda esta información definida, el LLM realiza una extracción inicial de entidades y relaciones. No obstante, seguía habiendo casos donde el LLM alucinaba u omitía información importante presente en el *chunk* original. Por ello, decidimos añadir una segunda iteración al proceso de extracción, buscando aumentar el *recall* en este sentido. Algunas aproximaciones que implementamos hasta obtener un resultado con el que estábamos satisfechos fueron las siguientes:

- **Validación directa sobre el JSON extraído:** La primera aproximación consistió en enviar

```

class PreysOnRel(BaseModel):
    source: str = Field(..., description="Name of the predator Species")
    target: str = Field(..., description="Name of the prey Species")
    description: str = Field("PREYS_ON", description="Indicates the consumption of ANY ANIMAL MATTER. This STRICTLY includes large preys, Insect, Fish, Crustacean, Squid, etc. Use PREYS_ON ONLY if the food is a living creature/animal.")

class FeedsOnRel(BaseModel):
    source: str = Field(..., description="Name of the Species")
    target: str = Field(..., description="Type of the Food Source")
    description: str = Field("FEEDS_ON", description="Represents the consumption of NON-ANIMAL food sources. This includes Grass, Leaves, Fruits, Seeds, Bark, Nectar and Aquatic Plants.")

```

Figura 2.5: Ejemplos de descripciones para las relaciones *PREYS_ON* y *FEEDS_ON* de nuestro grafo.

al LLM el JSON obtenido como resultado en la primera pasada junto con el esquema de datos completo. Sin embargo, el LLM no realizaba un proceso de razonamiento intermedio y se limitaba a replicar la extracción original, sin suponer un cambio significativo.

- **Identificación única de entidades y relaciones a añadir o eliminar:** Posteriormente, tras identificar que uno de los posibles problemas era pedir al LLM que volviese a generar un JSON con todas las entidades y relaciones finales, decidimos cambiar el enfoque y pedirle que únicamente nos devolviese aquellas que debían añadirse y eliminarse. Esta nueva aproximación permitió reducir la complejidad de la estructura de salida del modelo, pero no redujo las alucinaciones de la extracción original al no evaluar de forma razonada la validez de dichas entidades y relaciones.
- **Razonamiento sobre la extracción (*Chain of Thought*):** La solución definitiva consistió en mantener el enfoque del punto anterior y añadir un proceso de razonamiento previo y estructurado antes de dar la respuesta final. Con este propósito, definimos el esquema *Pydantic* de la figura 2.6 para poder obtener la salida de forma estructurada, y los *prompts* de la figura 2.7 que indican al modelo los pasos que debe seguir en este proceso de revisión. En estos *prompts* le indicamos al modelo los pasos que debe seguir: primero, le pedimos realizar un razonamiento dentro del campo “*thought_process*” fijándose en primer lugar en las alucinaciones, comprobando que cada dato extraído debe estar explícitamente en el texto, eliminándolo en caso contrario; posteriormente, debe revisar el texto para ver si hay algo que se haya omitido en la primera extracción, añadiéndolo si fuese necesario; y, por último, debe generar el JSON de salida con la información necesaria a añadir o eliminar, tomando su proceso de razonamiento como referencia.

Al incluir este proceso de razonamiento la mejora fue muy significativa, ya que ahora el modelo podía identificar qué cosas estaban presentes en el texto y no habían sido añadidas, así como aquellas que había añadido el modelo por su propio conocimiento implícito aunque no estuviesen representadas en el *chunk* procesado.

Una pequeña modificación que realizamos sobre esta aproximación, fue añadir un *fallback* en caso de que el razonamiento se extendiese demasiado y la API de Gemini cortase la salida por exceder el límite de tiempo o de *tokens* generados. Para ello, en los casos en los que detectábamos este error, se realizaba una segunda llamada al modelo con el mismo prompt, pero añadiendo la restricción de generar como máximo tres oraciones de razonamiento para las alucinaciones y otras tres para las omisiones (2. *REASONING BREVITY: Limit your 'thought_process' to a MAXIMUM of 3 sentences for hallucinations and 3 sentences for omissions.*). De esta forma,

aunque el proceso de revisión no es tan exhaustivo, podemos mantener parte de las ventajas que nos aporta esta extracción multi-paso.

2.3. Resolución de Entidades (*Entity Resolution*)

El siguiente paso a tener en cuenta en la creación y construcción del grafo es la resolución de entidades. En nuestro caso, hemos tratado de limitar este problema de diferentes formas en función de los tipos de entidades de nuestro grafo.

2.3.1. Resolución de entidades mediante *enums*

En la sección 2.2.1 detallamos como intentábamos reducir las alucinaciones del LLM en la extracción multi-paso utilizando el esquema *Pydantic* y los *enums* para la salida. Dado que en nuestro modelo de datos hay bastantes entidades que pueden tomar una serie de valores únicos ya definidos como la clase taxonómica (mamífero, ave, reptil, pez y anfibio), el tipo de reproducción (ovíparo, vivíparo u ovovivíparo) o su tipo de alimentación (carnívoro, herbívoro u omnívoro), tratamos de limitar los valores que podía extraer el LLM utilizando estos *enums* en nuestro esquema. Así, aunque en los textos puedan aparecer estos valores escritos de distintas formas, el LLM intentará ceñirse lo máximo posible a los *strings* predefinidos de los *enums*.

2.3.2. Resolución de entidades mediante descripciones

Para aquellas entidades que no podemos restringir a una cantidad limitada de valores, tratamos de definir claramente en las descripciones de nuestro esquema qué era lo que queríamos que extrajese el LLM (figura 2.8):

- **Hábitats (*HabitatNode*):** En el caso de las entidades que representan los hábitats, tratamos de diferenciarlas claramente de los lugares geográficos por medio de la descripción que le proporcionamos al LLM en el esquema, donde también incluíamos algunos ejemplos de la salida que esperábamos. No obstante, tras el proceso de generación del grafo identificamos que había ocasiones donde se habían creado nodos diferentes que representaban el mismo hábitat (por ejemplo *Desert* y *Deserts*), por lo que es uno de los posibles aspectos de mejora de cara a próximos hitos.
- **Ubicación Geográfica (*LocationNode*):** En el caso de las ubicaciones geográficas, el modelo se comporta de mejor manera, ya que en la descripción le indicamos que debe agrupar las localizaciones geográficas por países, continentes, mares u océanos. Además, como en el caso anterior, incluimos algunos ejemplos como por ejemplo, la “Selva Amazónica” debe resolverse como “Brasil” o “América del Sur”, y “Serengeti” como “Tanzania”.

2.3.3. Resolución dinámica de especies (*Species*) mediante LLM

Estas son las entidades fundamentales de nuestro grafo y era donde más variabilidad podíamos encontrarnos, ya que una especie concreta de animal podía venir representada de muchas formas en los distintos textos (nombre común, nombre científico, apodos, plurales, etc.). Además, dado que no íbamos a poder procesar un gran número de animales y especies concretas, nuestra intención era englobar todas las subespecies de un animal en su especie general, por ejemplo, en el caso de las serpientes, en lugar de tener muchas subespecies distintas, queríamos agruparlas dentro del nodo “serpiente”, tratando a su vez de priorizar que el grafo estuviese lo más interconectado posible y que no hubiese nodos *Species* que apenas tuviesen información.

Teniendo todos estos aspectos en cuenta, decidimos hacer una resolución dinámica de entidades utilizando un LLM. Para ello, llevamos un registro actualizado de todas las especies que se han añadido al grafo hasta el momento, de forma que al extraer de un *chunk* un nodo de tipo *Species* se siguen los siguientes pasos:

```

class GraphPatch(BaseModel):
    thought_process: str = Field(
        ...,
        description=(
            "You MUST structure your analysis in two parts: "
            "1. [HALLUCINATION CHECK]: Compare the INITIAL "
            "   EXTRACTION to the TEXT. Identify any entity or "
            "   relationship that is NOT explicitly written in the "
            "   text or contradicts it. "
            "2. [OMISSION CHECK]: Scan the TEXT for missing facts. "
            "   CRITICAL: You must ONLY look for missing info that "
            "   fits EXACTLY into our allowed schema categories (e.g "
            "   ., Habitat, DietType, Family). IGNORE general trivia "
            "   (like size, popularity, physical descriptions, or fun "
            "   facts). "
        )
    )
    entities_to_delete: List[EntityPointer] = Field(
        default=[],
        description="Entities from the initial extraction that MUST "
        "be deleted because they are NOT explicitly in the text."
    )
    relationships_to_delete: List[RelationshipPointer] = Field(
        default=[],
        description="Relationships from the initial extraction that "
        "MUST be deleted because they are NOT explicitly in the "
        "text."
    )
    missing_entities_to_add: MissingEntities = Field(
        default_factory=MissingEntities,
        description="NEW entities explicitly found in the text but "
        "missing from the initial extraction."
    )
    missing_relationships_to_add: MissingRelationships = Field(
        default_factory=MissingRelationships,
        description="NEW relationships explicitly found in the text "
        "but missing from the initial extraction."
    )

```

Figura 2.6: Esquema *Pydantic* definido para la revisión de la extracción inicial con el objetivo de reducir las alucinaciones y aumentar el *recall* de entidades y relaciones.

System prompt: “You are a Zoology Data Auditor. Your ONLY missions are to purge hallucinations and recover forgotten data from an initial extraction.

CRITICAL RULES:

1. REASON FIRST: You MUST use the 'thought_process' field to analyze the text before filling any lists. Structure your thoughts using [HALLUCINATION CHECK] and [OMISSION CHECK].
2. THE HALLUCINATION PURGE (DELETE): Every single entity and relationship in the INITIAL EXTRACTION must be explicitly backed by the text. If it relies on biological world knowledge or assumptions, YOU MUST DELETE IT.
3. BEWARE OF NEGATIONS & CONTRASTS: Pay extremely close attention to words like 'not', 'never', 'unlike', or 'while'. If the text states an animal DOES NOT do something, and the extraction says it does, DELETE IT.
4. SCHEMA-BOUND OMISSION (ADD): If a fact is clearly stated in the text but missing from the extraction, YOU CAN ONLY ADD IT IF IT FITS AN EXACT SCHEMA CATEGORY (e.g., 'Species', 'Habitat', 'Location').
5. ZERO DUPLICATES: Never add an omission if it (or a direct synonym) is already present in the INITIAL EXTRACTION. Check the list twice.
6. SCHEMA STRICTNESS: You must use the exact categories and relationship types provided in the schema.”

User prompt: f“Execute your audit on the data below.

- STEP 1: Perform the [HALLUCINATION CHECK]. Read the INITIAL EXTRACTION. If a data point is not explicitly in the TEXT, or contradicts it, mark it for deletion in your thought process.
- STEP 2: Perform the [OMISSION CHECK]. Read the TEXT. Find facts missing from the extraction and mark them for addition in your thought process.
- STEP 3: Generate the JSON patch using your thought process as a guide.

— TEXT —

text

— INITIAL EXTRACTION —

```
{initial_entities_json}  
{initial_relations_json}”
```

Figura 2.7: *System prompt* y *user prompt* diseñados para la segunda fase de extracción de entidades y relaciones (revisión).

```

class HabitatNode(BaseModel):
    type: str = Field(..., description="Type of natural environment
    or ecosystem: Forest, Desert, Grassland, Ocean, etc."
    "STRICT RULE: DO NOT output geographical locations, countries,
    continents, or specific map regions here. "
    "For example, NEVER output 'Africa', 'Amazon Rainforest', or '
    Spain'. If the text mentions a geographical location, use the
    LocationNode instead.")

class LocationNode(BaseModel):
    type: str = Field(..., description="The geographical location of
    the animal. "
    "STRICT RULE: You MUST generalize the location to a Country,
    Continent, Sea, or Ocean. "
    "Do NOT output specific regions, natural parks, or habitats. "
    "Examples: If the text says 'Amazon Rainforest', output 'Brazil'
    or 'South America'. "
    "If it says 'Serengeti', output 'Tanzania'. If it says 'Great
    Barrier Reef', output 'Pacific Ocean'.")

```

Figura 2.8: Esquema *Pydantic* para los nodos *HabitatNode* y *LocationNode*, tratando de diferenciarlos claramente mediante las descripciones.

1. **Validación sintáctica rápida:** En primer lugar se comprueba si el nombre extraído coincide exactamente (ignorando mayúsculas) con una especie ya presente en el grafo o si coincide al eliminar el sufijo de plural ("s"). En caso de que coincida, no se crea un nuevo nodo especie en el grafo y se utiliza el que ya ha sido creado.
2. **Evaluación semántica mediante LLM:** Si el primer paso de validación falla, el nuevo nombre extraído y la lista de especies recogidas en el grafo se envían al LLM en un *prompt* como el de la figura 2.9. En él, asignamos al modelo el rol de ser un experto en grafos de conocimiento con la tarea específica de resolver entidades. Para ello, debe comparar el nombre de la especie extraída con la lista de especies presentes en el grafo, decidiendo si corresponde a una entidad existente (*MATCH*), si representa una nueva entidad no vista hasta el momento (*NEW*) o si debe descartarse (*DISCARD*). Para ello, le hemos definido una serie de reglas para resolver entidades que puedan aparecer en plural o que puedan ser sinónimos (regla 1), generalizar las subespecies a la especie más general (regla 2), reconocer nuevas entidades (regla 3) y descartar entidades que realmente no representen una especie animal (reglas 4 y 5).

Finalmente, las entidades marcadas como "Discard" son eliminadas del flujo de inserción en el grafo, aquellas resueltas como "New" se añaden al grafo y al registro de especies, permitiendo que futuras extracciones de ese mismo animal o sus variantes se resuelvan localmente sin llamar al LLM y, por último, en aquellas que hay "Match" se utiliza la entidad que ya existía en el grafo.

2.4. Preprocesamiento para RAG

Durante esta fase de construcción automática del grafo, también realizamos el preprocesamiento necesario para preparar el sistema para la fase de recuperación. Concretamente, decidimos seguir la estrategia de generación de preguntas hipotéticas (*Hypothetical Questions*), ya que consideramos que se ajusta bastante a nuestro dominio, donde cada fragmento de texto contiene información muy concreta sobre una especie que puede relacionarse fácilmente con posibles preguntas de los usuarios, pudiendo anticiparnos de esta forma a sus intereses más frecuentes. Además, podemos cubrir un amplio rango

de preguntas, desde aquellas más científicas o detalladas sobre los datos del animal, hasta otras más generales relacionadas con comportamientos o curiosidades del mismo.

Para poder implementarla, después de insertar todas las entidades y relaciones extraídas al grafo, incluimos una nueva llamada al LLM con el *chunk* que estábamos procesando utilizando el *prompt* de la figura 2.10, donde le pedimos que genere un máximo de 10 preguntas para los fragmentos más largos y unas 5 preguntas para aquellos que sean más cortos y de los que no se pueda extraer tanta información. Como en todos los *prompts* utilizados en los pasos anteriores, le asignamos un rol concreto y le indicamos que su tarea es generar preguntas como si fuera un usuario haciendo preguntas sobre ese fragmento concreto, tratando de que algunas sean más factuales y otras cubran conceptos más amplios. Además, le indicamos que no debe hacer referencia del estilo “De acuerdo con el texto...” o “Basándome en la información proporcionada...”, ya que no son cosas que los usuarios vayan a incluir en sus consultas y, por tanto, podrían empeorar el funcionamiento del sistema.

Una vez tenemos la respuesta del LLM con la lista de preguntas asociadas al *chunk*, generamos una representación vectorial para cada una de ellas utilizando el mismo modelo de *embeddings* que usamos para los *chunks* en pasos anteriores, asegurando que todo esté mapeado en el mismo espacio vectorial. Cada una de las preguntas la almacenamos en nuestro grafo asociándola al fragmento correspondiente mediante la relación HAS_QUESTION, incluyendo en la entidad tanto la pregunta original en formato textual como su representación como *embedding*. A continuación se muestran algunos ejemplos de preguntas generadas:

- “What kind of animal is an iguana?”
- “What is the lion’s status in the African ecosystem?”
- “What is a group of lions called?”
- “How do insecticides impact woodpecker populations?”
- “Why do ducks turn their heads backward?”

Con toda esta información estructurada en el grafo, en la siguiente fase del proyecto podremos comparar el *embedding* de la consulta del usuario con las preguntas obtenidas para los distintos *chunks*, de forma que podamos identificar aquellas más similares para enriquecer el contexto del LLM con el fragmento de texto que permite responderlas.

System prompt: “You are an expert Knowledge Graph engineer and taxonomist working with a simplified, high-level animal ontology.

Your task is Entity Resolution. You will be given an extracted animal name and a canonical list of base animal entities.

Rules:

1. **SYNONYMS & EXACT MATCHES:** If the extracted name is a plural form (e.g., 'Snakes' -> 'Snake', 'Wolves' -> 'Wolf'), a known synonym, regional name, or refers to the same animal as one in the canonical list (e.g., 'Cougar' -> 'Mountain Lion', 'Orca' -> 'Killer Whale'), set status to 'MATCH' and output the exact matching name from the canonical list.
2. **GENERALIZATION (SUB-SPECIES TO PARENT):** If the extracted name is a specific type, breed, or sub-species of a broader generic animal that is already present in the canonical list, you **MUST** generalize it to the broader entity. Set status to 'MATCH'.
Examples:
 - 'Laysan Albatross' -> 'Albatross'
 - 'Emperor Penguin' -> 'Penguin'
 - 'Grizzly Bear' -> 'Bear'
3. **NEW ENTITIES:** If the extracted name is clearly an animal but represents a completely different animal lineage not covered by ANY broad category or synonym in the list, set status to 'NEW'. Provide the most standard, common English name for this new species in 'resolved_name'. Keep new names at a high, generic level if possible (e.g., output 'Eagle' instead of 'Bald Eagle').
4. **BROAD CLASSIFICATIONS & TRAITS (DISCARD):** If the extracted name is a dietary classification (e.g., 'Carnivore', 'Herbivore', 'Predator'), a broad taxonomic class (e.g., 'Mammal', 'Reptile', 'Bird', 'Amphibian'), or a generic biological family term rather than a specific animal (e.g., 'Feline', 'Canine', 'Big Cat'), you **MUST** discard it. Set status to 'DISCARD' and set 'resolved_name' to an empty string.
5. **NON-ANIMALS (DISCARD):** If the extracted name is NOT an animal (e.g., a plant, geographic location, inanimate object, person, or abstract concept), you **MUST** discard it. Set status to 'DISCARD' and set 'resolved_name' to an empty string.”

User prompt: f“Extracted Name: {extracted_name}

Canonical List of Species: {self.species_names}”

Figura 2.9: *System prompt* y *user prompt* definidos para la resolución de las entidades *Species* mediante un LLM

System prompt: “You are an expert in Retrieval-Augmented Generation (RAG) and your specific domain of expertise is Zoology. Your task is to generate hypothetical user search queries that are directly answered by the provided text. Think like a real user searching for information about animals.

CRITICAL: Never use meta-references like ‘According to the text’, ‘In this chunk’, or ‘Based on the provided information’. The questions must sound natural and independent.”

User prompt: f“Generate up to 10 hypothetical search questions that this specific chunk of text answers.

Rules:

1. Answerability: The exact answer to every question MUST be explicitly found within the chunk.
2. User Persona: Write the questions as a human would type them into a search bar (concise, direct, and natural).
3. Diversity: Mix specific factual questions with broader conceptual questions if applicable.
4. Volume: If the chunk contains dense information, return up to 10. If it is sparse or short, return up to 5.

Chunk: {text}”

Figura 2.10: *System prompt* y *user prompt* definidos para la generación de preguntas hipotéticas a partir de un *chunk*.

3. Hito 3: Implementación y Evaluación de *Retrievers*

3.1. Retriever Text2Cypher

Para la implementación de este *retriever*, hemos tomado como base la plantilla del repositorio de referencia y nos hemos centrado en adaptarla a nuestro dominio y refinar el comportamiento del LLM utilizado para que se adaptase a nuestras necesidades, con el objetivo de asegurar que fuera capaz de traducir las preguntas del usuario en lenguaje natural a consultas Cypher precisas con las que enriquecer el contexto de la respuesta final. Para ello, nos hemos enfocado en incluir ejemplos en el *prompt*, siguiendo la técnica *few-shot*, que puedan servir como referencia al LLM. Además, hemos creado un mapa terminológico para que tenga claro cómo traducir ciertos elementos del lenguaje natural a nuestro dominio específico y hemos definido reglas estrictas en el *system prompt* para que se ajuste al comportamiento esperado.

Con la intención de enseñarle al modelo las estructuras más comunes de nuestro grafo para que pueda tomarlas como referencia a la hora de generar consultas, creamos algunos ejemplos donde cubrimos diferentes entidades y relaciones del modelo de datos:

- Ejemplo de recuperación directa de propiedades:

```
text2cypher.add_few_shot_example(  
    "What is the top speed and maximum weight of a Lion?",  
    "MATCH (s:Species {name: 'Lion'}) RETURN s.top_speed_kmh, s.  
        weight_max_kg"  
)
```

Figura 3.1: Ejemplo de recuperación directa de propiedades para obtener la máxima velocidad y peso de un león.

- Consultas que requieren relacionar nodos:

```
text2cypher.add_few_shot_example(  
    "What type of diet do tigers have and what do they feed on?",  
    "MATCH (s:Species {name: 'Tiger'})-[:HAS_DIET_TYPE]->(d:DietType  
        ), (s)-[:PREYS_ON]->(f:FoodSource) RETURN d.type, f.type"  
)
```

Figura 3.2: Ejemplo de recuperación que requiere relacionar nodos, donde se pide el tipo de dieta tienen los tigres y de qué se alimentan.

- Uso de condiciones lógicas:

```

text2cypher.add_few_shot_example(
    "Which animals have a lifespan greater than 50 years?",
    "MATCH (s:Species) WHERE s.lifespan_years > 50 RETURN s.name, s.lifespan_years"
)

```

Figura 3.3: Ejemplo de recuperación haciendo uso de condiciones lógicas para obtener animales con una esperanza de vida superior a 50 años.

- Ejemplos específicos para enseñar al modelo cuándo usar la relación de caza (PREYS_ON para carnívoros) frente a la de alimentación general (FEEDS_ON para herbívoros) o la relación de localizaciones geográficas (FOUND_IN) frente a la relación de los hábitats (INHABITS):

```

# PREYS ON & FEEDS ON
text2cypher.add_few_shot_example(
    "What does a chimpanzee eat?",
    "MATCH (s:Species {name: 'Chimpanzee'}) OPTIONAL MATCH (s)-[:PREYS_ON]->(prey:Species) OPTIONAL MATCH (s)-[:FEEDS_ON]->(food:FoodSource) RETURN s.name AS species, collect(DISTINCT prey.name) AS preys_on, collect(DISTINCT food.type) AS feeds_on"
)

# FOUND IN VS INHABITS
text2cypher.add_few_shot_example(
    "In which countries or regions are kangaroos found?",
    "MATCH (s:Species {name: 'Kangaroo'})-[:FOUND_IN]->(l:Location) RETURN l.type"
)

text2cypher.add_few_shot_example(
    "What kind of ecosystem or biome does the polar bear inhabit?",
    "MATCH (s:Species {name: 'Polar Bear'})-[:INHABITS]->(h:Habitat) RETURN h.type"
)

```

Figura 3.4: Ejemplos de uso específicos para las relaciones de alimentación y localización de los animales.

Además de incluir ejemplos en el *prompt*, para resolver las ambigüedades del lenguaje natural también hemos incluido un diccionario con términos (figura 3.5) que sirva de guía al LLM para escoger correctamente las entidades y relaciones a la hora de construir las consultas. Hemos diseñado este mapa de términos para que sea escalable, por lo que podríamos añadir nuevas reglas si fuese necesario al detectarse algún problema durante la evaluación. Algunas de las distinciones más importantes que le enseñamos son:

- Dieta: Asociar verbos como “cazar” o “matar” con la relación PREYS_ON hacia otros animales, y verbos como “pastar” o “alimentarse de plantas” con la relación FEEDS_ON hacia fuentes de alimento no animales.
- Ubicación vs Hábitat: Diferenciar entre ubicaciones geográfico-políticas (países o continentes, usando FOUND_IN) y ecosistemas naturales o biomas (usando INHABITS).

```

text2cypher.add_terminology_map(
    "animal, creature, species",
    "Refers to the node with label (:Species)"
)

text2cypher.add_terminology_map(
    "what does X eat, what do X eat, diet of X, food of X. When
    asking what an animal eats, check BOTH relationships:"
    "[:PREYS_ON]->(:Species) for animal prey AND [:FEEDS_ON]->(:
    FoodSource) for non-animal food sources. Use OPTIONAL MATCH
    for both and return all results."
)

text2cypher.add_terminology_map(
    "country, continent, geographic region, area, located in",
    "Use the relationship [:FOUND_IN]->(:Location) to refer to
    specific geographical and political places."
)

text2cypher.add_terminology_map(
    "biome, ecosystem, type of environment, terrain, inhabits",
    "Use the relationship [:INHABITS]->(:Habitat) to refer to the
    natural biome or habitat type."
)

text2cypher.add_terminology_map(
    "mentioned in the text, according to the documents, sources say"
    ,
    "Traverse from (:Species)<-[:HAS_ENTITY]-(:Chunk)<-[:HAS_CHUNK
    ]-(Document)"
)

```

Figura 3.5: Mapas terminológicos.

Finalmente, todos estos ejemplos y definiciones se envían al modelo mediante un *system prompt* como el de la figura 3.6. Además de esta información, también se incluye el esquema de nuestra base de datos para que tenga acceso a todas las entidades y relaciones existentes. Asimismo, hemos decidido incorporar los *enums* con los posibles valores para algunas entidades que habían sido definidos previamente al crear automáticamente el grafo (como la clase a la que pertenecen los animales o su estado de conservación), con el objetivo de limitar posibles errores de escritura a la hora de construir la consulta. Por último, añadimos una serie de reglas para especificar claramente cómo debe ser la salida y, sobre todo, hacer hincapié en que se ajuste al esquema proporcionado y en que complete los valores utilizando *title case* para aquellas propiedades que no cuenten con un enumerado predefinido.

En las pruebas que hemos realizado hasta ahora, este *retriever* nos ha funcionado relativamente bien con la implementación actual. No obstante, durante la fase evaluación formal del sistema analizaremos más en detalle su rendimiento para ver si es necesario hacer algún ajuste adicional en el *prompt*, los ejemplos o los mapas terminológicos.

3.2. Retriever RAG Vectorial

Para la recuperación semántica de la información, hemos optado por un enfoque híbrido, combinando la búsqueda vectorial sobre las preguntas que habíamos generado durante el proceso de construcción automática del grafo con una búsqueda tradicional por palabras clave (*full-text*). Nuestro objetivo al

System prompt: “You are an Information Retrieval Agent operating a Neo4j Knowledge Graph. Your task is to translate natural language questions into exact Cypher queries to extract the required data.

— GRAPH SCHEMA —
{schema_str}

— TERMINOLOGY MAP — Use the following domain-specific terminology mappings to understand the user’s intent:
{terminology_str}

— ALLOWED ENUM VALUES (STRICT EXACT MATCH) —
{enums_str}

— FEW-SHOT EXAMPLES — Use these examples as a reference for the expected Cypher structure:
{examples_str}

— FORMATTING INSTRUCTIONS —

1. Return ONLY the raw Cypher query.
2. Do NOT wrap the query in markdown code blocks (e.g., no “cypher or “”).
3. Do NOT provide any explanations, apologies, or conversational text before or after the query.
4. Ensure the query is syntactically correct and optimized for Neo4j.
5. Use EXACTLY the node labels, relationship types, and properties provided in the schema and terminology map. Do not hallucinate properties.
6. GENERAL TITLE CASE RULE: For ALL OTHER nodes and string properties not listed in the enums (such as Species 'name', Location 'type', etc.), you MUST format the search values in Title Case (e.g., use 'African Elephant' instead of 'african elephant', or 'South Africa' instead of 'south africa').

Figura 3.6: *System prompt* definido para generar consultas Cypher a partir de una consulta del usuario.

utilizar ambos métodos a la vez es aprovechar las ventajas que aporta cada uno de ellos para poder recuperar un conjunto de fragmentos que permita enriquecer el contexto del LLM y responder al usuario de la mejor forma posible.

3.2.1. Recuperación vectorial (*Vector Retriever*)

En la sección 2.4 nos encargamos de construir la base necesaria para implementar esta recuperación vectorial de la forma que teníamos en mente, generando para cada fragmento una serie de preguntas que dicho *chunk* era capaz de responder (*hypothetical questions*) y que utilizaríamos para compararlas con la consulta del usuario.

Gracias a este preprocesamiento, los pasos que se siguen para llevar a cabo esta recuperación son los siguientes: cuando el usuario realiza una consulta, el sistema genera un *embedding* de la misma utilizando el mismo modelo con el que se indexaron las preguntas originales, asegurando que ambas representaciones estén mapeadas en el mismo espacio vectorial. A continuación, se calcula la similitud y se extraen los k resultados más afines (en principio hemos definido $k = 15$ para tener un gran número de candidatos y filtrar en pasos posteriores). Además, para optimizar la búsqueda, creamos un índice

específico sobre las distintas preguntas generadas para cada *chunk*.

3.2.2. Recuperación por palabras clave (*Full-Text Retriever*)

A diferencia del *retriever* vectorial, en este caso la comparación se realiza directamente contra el texto de los *chunks*, buscando coincidencias exactas de las palabras clave de la pregunta del usuario. Para extraer estos términos más relevantes de la consulta del usuario, decidimos utilizar un LLM en lugar de aplicar técnicas tradicionales de PLN (como la eliminación de *stop-words* o la lematización), ya que consideramos que estos modelos son capaces de comprender las relaciones semánticas de la frase e identificar con precisión cuáles son las entidades realmente importantes para realizar la posterior recuperación. Para ello, definimos el *prompt* de la figura 4.2 y, una vez el modelo nos devuelve las palabras clave, lanzamos la consulta contra nuestro índice de texto y recuperamos nuevamente los 15 mejores resultados que filtraremos en el siguiente paso.

System prompt: “You are a precise keyword extraction assistant for an Animal Knowledge Graph. Your ONLY task is to extract the core search terms from a natural language query to be used in a Full-Text search.

RULES:

1. Remove all stop words (e.g., 'what', 'are', 'the', 'of', 'in', 'is', 'how', 'tell', 'me', 'about', etc.).
2. Keep essential nouns, adjectives, and proper names (e.g., species names, habitats, diets, geographic locations).
3. Return ONLY a single line of space-separated keywords.
4. Do NOT add any conversational text, explanations, or quotes.

EXAMPLES:

User: “What are the natural predators of the African Elephant?”

Output: natural predators African Elephant

User: “Tell me about the habitat and diet of carnivorous reptiles.”

Output: habitat diet carnivorous reptiles

User: “Why do ducks turn their heads backward?”

Output: ducks heads backward

Figura 3.7: *System prompt* definido para la extracción de las palabras clave de la consulta del usuario.

3.2.3. Recuperador híbrido y Reranking (*Hybrid Retriever*)

Una vez tenemos todos los resultados de los dos recuperadores anteriores, el siguiente paso es decidir cuáles de ellos son los mejores para responder a la pregunta inicial del usuario. Para ello, lo primero que hemos hecho ha sido fusionarlos, pero como ambos *retrievers* devuelven los fragmentos con una puntuación que está en escalas diferentes, realizamos una normalización *min-max* para tener todos los *scores* dentro del rango [0, 1]. Tras esta normalización, combinamos los resultados ponderando su puntuación en función del *retriever* que los haya recuperado, concretamente damos un 70 % de peso a los *chunks* procedentes de la búsqueda vectorial y un 30 % a los de la búsqueda por palabras clave. Nuestro objetivo a la hora de ponderar los resultados es dar más importancia a esta similitud semántica, ya que entendemos que será capaz de recuperar fragmentos más relevantes en relación al significado global de la consulta. No obstante, mantenemos un 30 % de peso para la búsqueda por

palabras clave porque puede resultar útil en casos donde aparezcan términos específicos, nombres propios o coincidencias exactas que la búsqueda semántica podría no priorizar, logrando de esta forma un equilibrio entre la comprensión contextual y la precisión léxica.

Después de combinar todos los resultados, volvemos a tomar los k mejores (15 por defecto) y hacemos un último paso de *reranking* con el modelo *bge-reranker-v2-m3* ¹. Finalmente, reordenamos la lista de resultados en base a esta puntuación obtenida por el modelo de *reranking* y devolvemos únicamente los 5 mejores, tratando de proporcionar al LLM el mejor contexto posible sin que sea demasiado extenso.

3.3. Retriever *Queries* Manuales

Este es el *retriever* más sencillo de los tres que componen el sistema, pero resulta de gran utilidad para mejorar su comportamiento con consultas que pueden ser complejas pero recurrentes dentro de nuestro dominio. Gracias a este enfoque, en lugar de depender de que el LLM genere consultas Cypher a partir de lenguaje natural (como ocurre en el *retriever* *text2cypher*), el sistema agéntico puede identificar cuándo la pregunta del usuario se ajusta a alguna de las consultas predefinidas y ejecutarla directamente sobre la base de datos. Esto nos permite obtener respuestas de forma más rápida y fiable, garantizando que los resultados recuperados sean exactamente los que nosotros esperamos y evitando posibles errores a la hora de generar estas consultas complejas con un LLM.

En la figura 3.8 se muestran algunos ejemplos de las consultas que hemos implementado hasta el momento. No obstante, a medida que vayamos desarrollando y evaluando el sistema, añadiremos nuevas consultas para cubrir casos que no hayamos considerado hasta el momento.

¹<https://huggingface.co/BAAI/bge-reranker-v2-m3>

```

"species_full_profile": ""
MATCH (s:Species {name: $species_name})
OPTIONAL MATCH (s)-[:BELONGS_TO_CLASS]->(c:AnimalClass)
OPTIONAL MATCH (s)-[:MEMBER_OF_FAMILY]->(f:Family)
OPTIONAL MATCH (s)-[:HAS_CONSERVATION_STATUS]->(cs:
    ConservationStatus)
OPTIONAL MATCH (s)-[:HAS_SKELETAL_STRUCTURE]->(ss:
    SkeletalStructure)
OPTIONAL MATCH (s)-[:REPRODUCES_VIA]->(rm:ReproductionMethod)
OPTIONAL MATCH (s)-[:HAS_ACTIVITY_CYCLE]->(ac:ActivityCycle)
OPTIONAL MATCH (s)-[:HAS_DIET_TYPE]->(d:DietType)
OPTIONAL MATCH (s)-[:FEEDS_ON]->(fs:FoodSource)
OPTIONAL MATCH (s)-[:PREYS_ON]->(prey:Species)
OPTIONAL MATCH (s)-[:ORGANIZED_IN]->(soc:SocialStructure)
OPTIONAL MATCH (s)-[:LIVES_IN_ENVIRONMENT]->(env:EnvironmentType
)
OPTIONAL MATCH (s)-[:INHABITS]->(hab:Habitat)
OPTIONAL MATCH (s)-[:FOUND_IN]->(loc:Location)
OPTIONAL MATCH (s)-[m:MIGRATES_TO]->(mig_loc:Location)
RETURN s.name AS Species,
        collect(DISTINCT c.type) AS Class,
        collect(DISTINCT f.type) AS Family,
        (...)
        s.weight_max_kg AS MaxWeight,
        s.top_speed_kmh AS TopSpeed,
        s.lifespan_years AS Lifespan
        "",

"endangered_by_environment": ""
MATCH (s:Species)-[:LIVES_IN_ENVIRONMENT]->(e:EnvironmentType {
    type: $environment_name})
MATCH (s)-[:HAS_CONSERVATION_STATUS]->(c:ConservationStatus)
WHERE c.type IN ['CR (Critically Endangered)', 'EN (Endangered)
']
RETURN s.name, c.type, e.type
ORDER BY c.type
        "",

"predator-prey_chain": ""
MATCH (s:Species {name: $species_name})
OPTIONAL MATCH (predator:Species)-[:PREYS_ON]->(s)
OPTIONAL MATCH (s)-[:PREYS_ON]->(prey:Species)
OPTIONAL MATCH (s)-[:FEEDS_ON]->(food:FoodSource)
RETURN
    s.name AS species,
    collect(DISTINCT predator.name) AS hunted_by,
    collect(DISTINCT prey.name) AS hunts,
    collect(DISTINCT food.type) AS feeds_on
    "",

"social_structure_by_class": ""
MATCH (s:Species)-[:BELONGS_TO_CLASS]->(c:AnimalClass {type:
    $class_name})
MATCH (s)-[:ORGANIZED_IN]->(ss:SocialStructure)
RETURN ss.type AS social_structure, count(s) AS species_count
ORDER BY species_count DESC
""

```

Figura 3.8: Ejemplos de consultas manuales para el sistema agéntico.

4. Hito 4: Orquestación Agéntica y Evaluación Final

4.1. Sistema Agéntico

Una vez definidos e implementados los *retrievers* de manera independiente, el siguiente paso consistió en integrarlos dentro de un sistema agéntico. El objetivo de esta fase del proyecto era desarrollar un sistema capaz de procesar consultas introducidas por el usuario en lenguaje natural para decidir qué herramienta o herramientas son las más adecuadas para recuperar un contexto con el cuál poder responderla. Para ello, hemos tomado como punto de partida el sistema implementado en el repositorio de referencia y hemos introducido las mejoras y adaptaciones necesarias para poder cubrir nuestros casos de uso.

Antes de comentar los detalles de la arquitectura del sistema, cabe destacar que las primeras pruebas las realizamos utilizando modelos en local a través de Ollama¹, concretamente de la familia Gemma 4². Esto nos permitió depurar el comportamiento de cada uno de los componentes de forma independiente pero, al integrar todo el sistema, no estábamos del todo satisfechos con la latencia a la hora de responder ni con la capacidad de razonamiento de los distintos agentes. Por ello, decidimos pasar a utilizar modelos de pago a través de distintas APIs. Por un lado, hicimos algunas pruebas cualitativas con el modelo *Llama 3 70B Versatile* utilizando la API de Groq³ pero el límite de tokens diarios del *tier* gratuito nos llevó a utilizar de nuevo el modelo *Gemini 2.5 Flash Lite*⁴ para terminar de construir el sistema y realizar la evaluación del mismo (sección 4.2), tal y como hicimos en el proceso de construcción automática del grafo (capítulo 2).

En las siguientes secciones explicaremos detalladamente cada uno de los componentes que forman parte del sistema agéntico que hemos implementado y cuyo esquema simplificado puede verse en la figura 4.1.

4.1.1. El Agente Enrutador (*Router*)

El primer agente de nuestro sistema es el **Agente Enrutador**, que tiene como función principal evaluar la intención semántica de la consulta del usuario para decidir qué estrategia de recuperación de contexto debe aplicarse. Para ello, cuenta con una serie de herramientas, cuyas descripciones se han detallado en el *system prompt*, y que le permiten decidir qué proceso es necesario para responder la consulta del usuario. Estas herramientas se pueden clasificar en tres bloques según su propósito:

- **Inputs conversacionales:** En este grupo se encuentran las herramientas **greetings**, **skills** y **out_of_scope**, encargadas de generar una respuesta directa para contestar al usuario que ha mostrado una intención conversacional. La primera de ellas, **greetings**, será utilizada cuando el usuario introduzca un saludo que será respondido con uno de los textos definidos en la figura (—————METER FIGURA DE SALUDOS Y EN LA MISMA METER TAMBIEN LOS TEXTOS DE RESPUESTA DE SKILLS Y OUT OF SCOPE—————) elegido de forma aleatoria. Por su parte, la herramienta **skills** se seleccionará cuando el usuario pregunte acerca de

¹<https://ollama.com/>

²<https://ollama.com/library/gemma4>

³<https://groq.com/>

⁴<https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/2-5-flash-lite?hl=es>

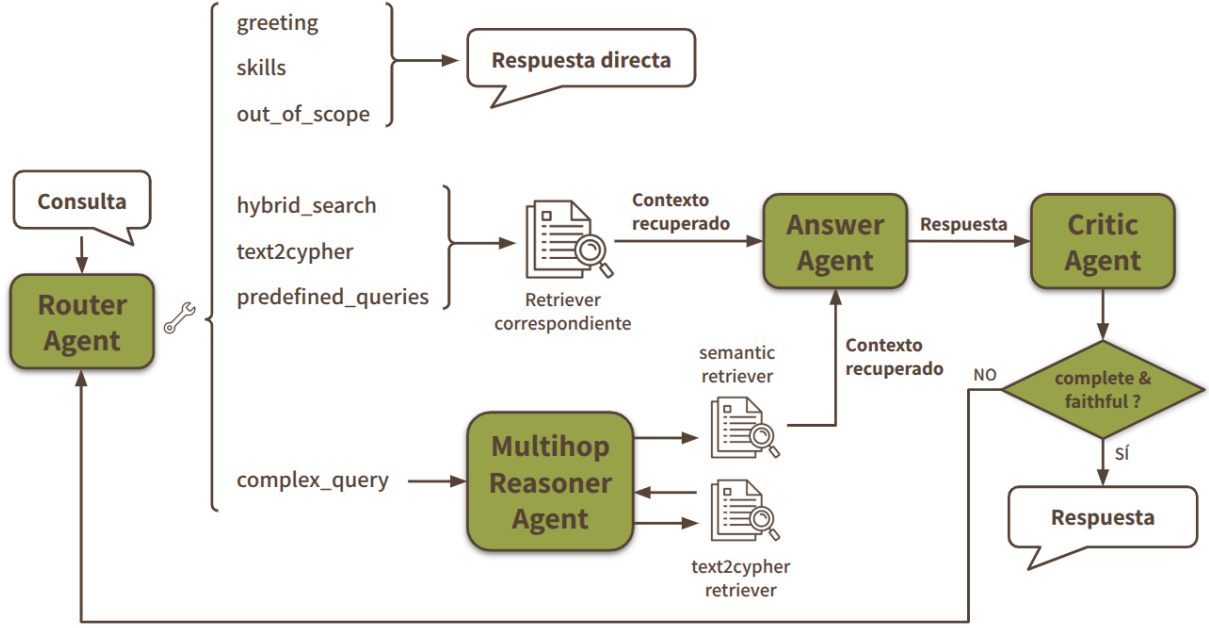


Figura 4.1: Esquema simplificado del sistema agéntico completo.

las capacidades del sistema y `out_of_scope` cuando la consulta del usuario se encuentre fuera del dominio zoológico.

- **Estrategias de Recuperación Dinámica:** Constituyen el núcleo del sistema RAG. Se dividen en búsqueda híbrida (`hybrid_search`) para consultas cualitativas o descriptivas sobre texto no estructurado, y consulta estructurada (`text2cypher`) para interactuar directamente con el grafo de conocimiento mediante lenguaje Cypher cuando se requieren datos exactos o agregaciones. Asimismo, se incluye el modo de consulta compleja (`complex_query`) para preguntas de múltiples saltos que combinan ambos mundos.
- **Consultas Predefinidas (*Predefined Queries*):** Para optimizar el rendimiento en escenarios comunes del dominio biológico, se han implementado herramientas de recuperación directa. Estas permiten extraer perfiles completos de especies, identificar animales en peligro según su medio ambiente, analizar cadenas tróficas o estudiar estructuras sociales según la clase taxonómica.

$GREETING_{RESPONSES} = [ElBúhoSabiondo("Hoothoot! I am a highly intellectual knowledge assistant perched on a branch")]$

El Perro Hiperactivo ("WOOF! Hi! Hello! I am a VERY GOOD knowledge assistant specialized in the animal kingdom! You can throw questions at me about our taxonomy, behavioral traits (like fetching!), diet (SQUIRREL?!), habitats, and conservation status!"),

El Gato Arrogante ("Meow. I am your supreme feline overlord, but for now, I'll act as a knowledge assistant for the animal kingdom. You may grovel and ask me about taxonomy, physical traits, my preferred premium diet, habitats (mainly cardboard boxes), and conservation status."),

El Perezoso Relajado ("Yawn... Helloooo... I am a knowledge assistant... eventually... specialized in the animal kingdom. Take your time asking me about our taxonomy, traits, diet (mostly leaves), habitats, and conservation status... no rush..."),

El Capibara Chill ("Sup. I am the chilliest knowledge assistant in the animal kingdom. Grab a spot in the hot spring and ask me about our taxonomy, physical or behavioral traits, diet, habitats, and conservation status. We're all friends here."),

El Pingüino Formal ("Greetings! Please excuse my tuxedo, I am a very formal knowledge assistant specialized in the animal kingdom. You may inquire about our taxonomy, physical or behavioral traits, diet (strictly seafood), freezing habitats, and conservation status."),

El Delfín Entusiasta (.Eee-eee! *Splash* I am a super-smart, echolocating knowledge assistant riding the waves of the animal kingdom! You can click and squeak at me about our taxonomy, behavioral traits, diet, marine habitats, and conservation status!"),

El Camaleón Camuflado ("Hello... Now you see me, now you don't! I am a highly adaptable knowledge assistant blending into the animal kingdom. You can ask me to reveal facts about taxonomy, physical traits (like my fabulous colors), diet (mostly crunchy bugs), habitats, and conservation status."),

La Abeja Adicta al Trabajo ("Bzzzz! Welcome to the hive! I am a very busy worker-bee knowledge assistant pollinating the animal kingdom. Don't sting me with hard questions, but you can ask about taxonomy, behavioral traits, diet (sweet, sweet nectar), habitats, and conservation status!")]

oUTOFSCOPE_RESPONSE = ("Umm...moo?Sorry,thisquestioniswayoutsidemypasture.""Iamdesignedtoche

sKILLS_RESPONSE = ("Withmyeightarmsandthreebrains,Icanjuggledetailedquestionsaboutspectificanima

Lógica de Decisión y Reglas de Enrutamiento

El comportamiento del Agente Enrutador está regido por un conjunto de reglas deterministas integradas en su instrucción principal. El agente aplica una lógica secuencial priorizando, en primer lugar, el control de la interacción. Si una consulta es un saludo o está fuera de dominio, la herramienta correspondiente se invoca de manera aislada, garantizando una respuesta rápida y coherente.

Para las consultas que requieren conocimiento, el sistema prioriza el uso de las herramientas predefinidas siempre que la intención del usuario encaje en sus parámetros. Si la pregunta requiere una generación más dinámica, el agente evalúa su naturaleza: asocia explicaciones conceptuales a la búsqueda híbrida y requerimientos factuales a la traducción *text-to-Cypher*.

Finalmente, cabe destacar la versatilidad del Agente Enrutador para gestionar consultas compuestas. El sistema está diseñado de tal forma que, si el usuario formula varias preguntas encadenadas en una misma interacción, el agente es capaz de desagregar las distintas intenciones y orquestar la ejecución de múltiples herramientas de forma simultánea para responder a cada una de ellas. Por otro lado, cuando la consulta presenta una alta complejidad y requiere un razonamiento multipaso —es decir, escenarios donde la resolución de una pregunta condiciona el contexto de la siguiente—, el enrutador selecciona la herramienta **complex_query**. Esta acción transforma al enrutador en un supervisor que delega el flujo de ejecución a un agente secundario especializado en razonamiento iterativo, cuyo diseño y funcionamiento se detallarán en una sección posterior de este documento.

Una vez que el Agente Enrutador determina qué herramientas deben ejecutarse, el sistema procede a su invocación. Si la decisión consiste en emitir un mensaje de control o emplear un único *retriever* estándar, la ejecución es inmediata. Sin embargo, cuando la naturaleza de la pregunta exige un razonamiento iterativo y la herramienta seleccionada es **complex_query**, el control del flujo se delega a un componente especializado cuyas características se detallan a continuación.

4.1.2. Agente Razonador Multipaso (*Multi-hop Reasoner*)

Este agente interviene de manera exclusiva en escenarios analíticos complejos donde la respuesta final no puede extraerse en una única consulta, sino que requiere la obtención secuencial de resultados intermedios. Su arquitectura se divide en dos fases bien diferenciadas: una fase de planificación cognitiva y una fase de ejecución dinámica.

System prompt: “You are an expert routing assistant for a zoology and animal biology knowledge base. Your job is to analyze the user’s query and select the most appropriate tool or combination of tools to handle it.

Available tools and their descriptions:
{tools_str}

ROUTING RULES — Apply strictly in the following order:

1. greeting
 - Use when the message is conversational and needs NO knowledge lookup
 - Examples: “Hello”, “Hi”, “Good morning”, “Thanks”, “Goodbye”, “Who are you?”.
 - Do NOT use this if the user asks about what the system can do.
 - NEVER combine with other tools.
2. Keep essential nouns, adjectives, and proper names (e.g., species names, habitats, diets, geographic locations).
3. Return ONLY a single line of space-separated keywords.
4. Do NOT add any conversational text, explanations, or quotes.

EXAMPLES:

User: “What are the natural predators of the African Elephant?”

Output: natural predators African Elephant

User: “Tell me about the habitat and diet of carnivorous reptiles.”

Output: habitat diet carnivorous reptiles

User: “Why do ducks turn their heads backward?”

Output: ducks heads backward

Figura 4.2: *System prompt* definido para la extracción de las palabras clave de la consulta del usuario.

Fase de Planificación

La primera tarea del agente no es recuperar información, sino trazar una estrategia. Mediante un *system prompt* específico, se instruye al modelo de lenguaje para actuar como un Planificador de Consultas (*Query Planner*). Su función es descomponer la pregunta del usuario en un plan de acción estrictamente secuencial, utilizando únicamente dos herramientas base: la consulta estructurada (`text2cypher`) y la búsqueda semántica pura (`semantic_search`).

La heurística principal dictada al modelo establece que siempre debe reducir primero el espacio de búsqueda. Por ejemplo, ante la pregunta “*De las especies que habitan en la sabana, ¿cuáles son sus estrategias de supervivencia?*”, el planificador no busca estrategias de supervivencia de forma global, sino que define un Paso 1 para listar mediante Cypher los animales de la sabana, y un Paso 2 para buscar semánticamente las estrategias de ese subconjunto específico.

Motor de Ejecución y Gestión del Contexto

Una vez trazado el plan, un motor de ejecución orquesta los pasos, garantizando que la salida de una etapa actúe como filtro o contexto de la siguiente. El flujo de trabajo habitual de este motor opera de la siguiente manera:

1. **Recuperación Estructurada y Extracción de Entidades:** El sistema ejecuta en primer

lugar la instrucción `text2cypher`. El resultado de esta consulta al grafo se almacena como contexto estructurado y, de manera simultánea, un algoritmo extrae dinámicamente todas las entidades resultantes (generalmente nombres de especies) acumulándolas en un conjunto de validación.

2. **Búsqueda Vectorial Paralelizada:** Al llegar al paso de `semantic_search`, el sistema realiza una intersección entre las entidades extraídas en el paso anterior y aquellas presentes en la base de datos vectorial. Para cada entidad válida, se lanza una consulta semántica individual. Con el objetivo de minimizar la latencia, estas múltiples búsquedas se ejecutan de manera concurrente mediante un *pool* de hilos (`ThreadPoolExecutor`).

Optimizaciones de Rendimiento y Latencia

Para garantizar la viabilidad del sistema en tiempo de ejecución y evitar la saturación de la ventana de contexto (*context window*) del LLM sintetizador final, se han implementado tres decisiones de diseño críticas en esta fase:

- **Truncamiento Topológico:** En caso de que el primer paso devuelva un volumen inmanejable de entidades, el sistema ordena y limita la búsqueda a un máximo de 10 especies. Este filtrado no es aleatorio; se realiza priorizando aquellas entidades con mayor grado de conectividad topológica en el grafo, bajo la premisa de que los nodos más interconectados albergan una mayor riqueza de información en el dominio zoológico.
- **Limitación de Fragmentos (*Top-k*):** Para cada entidad buscada concurrentemente, la recuperación se restringe al fragmento de texto (*chunk*) más relevante ($Top-k = 1$), asegurando un contexto denso y conciso.
- **Omisión de la Búsqueda Híbrida:** A diferencia de los *retrievers* estándar del sistema, en el razonador multipaso se prescinde de la técnica de búsqueda híbrida. Dado que el espacio de búsqueda ya se encuentra acotado explícitamente a un conjunto de entidades filtradas por el grafo, invocar a un LLM adicional para la extracción de palabras clave aportaría una latencia innecesaria sin un incremento justificable en la precisión de recuperación.

El resultado final de esta orquestación es un contexto enriquecido y altamente focalizado que combina datos estructurados del grafo con literatura especializada, listo para ser procesado por el agente generador final.

4.1.3. Generación de la Respuesta (*Synthesizer Agent*)

Una vez que el sistema ha recuperado y consolidado el contexto relevante —ya sea mediante una invocación directa a un *retriever* estándar o tras el procesamiento complejo del Agente Razonador Multipaso—, la información se transfiere al Agente Generador. Este componente tiene la responsabilidad exclusiva de redactar la respuesta final que se entregará al usuario.

Para asegurar el rigor científico del documento y evitar el fenómeno de la alucinación, el *system prompt* de este agente le impone restricciones estrictas:

- **Cero conocimiento previo:** Debe formular la respuesta basándose única y exclusivamente en el contexto recuperado, ignorando los datos preentrenados del modelo base.
- **Precisión y concisión:** La redacción debe ser directa, omitiendo frases de relleno (como “*Según el texto...*”) y manteniéndose, idealmente, en la extensión de un único párrafo.
- **Trazabilidad:** Cada afirmación o dato factual introducido en la respuesta debe ir acompañado de una cita en línea indicando el fragmento (*chunk*) exacto del que procede.
- **Manejo de vacíos de información:** Si el contexto proporcionado no contiene los datos necesarios para resolver la consulta, el agente tiene prohibido inferir una respuesta, debiendo emitir

un mensaje de contingencia predefinido ("*This information is not in the knowledge base*").

4.1.4. Evaluación Continua: Agente Crítico y Ciclo de Refinamiento

Con el fin de garantizar que el resultado del Agente Generador cumple con los estándares de calidad esperados, la respuesta propuesta no se devuelve inmediatamente al usuario, sino que se somete al escrutinio de un **Agente Crítico** (*Critique Agent*). Este evaluador analiza la tríada formada por la pregunta original, el contexto recuperado y la respuesta generada, validando dos dimensiones fundamentales:

1. **Fidelidad (*Faithfulness*)**: Verifica que absolutamente todas las afirmaciones presentes en la respuesta estén respaldadas explícitamente por el contexto, penalizando cualquier salto lógico o deducción externa.
2. **Complejidad (*Completeness*)**: Comprueba que la respuesta aborda de forma exhaustiva todas las aristas y subpreguntas planteadas por el usuario.

El Agente Crítico devuelve su evaluación estructurada bajo un esquema de datos formal (JSON) que incluye campos booleanos para estas dos dimensiones, así como retroalimentación justificativa.

Bucle Iterativo de Mejora (*Feedback Loop*)

La arquitectura incorpora un mecanismo de resiliencia basado en las evaluaciones del crítico. Si la respuesta generada es considerada infiel o incompleta, el sistema inicia un ciclo iterativo de refinamiento. En este escenario, el evaluador no solo señala el fallo, sino que extrae específicamente la información faltante (*missing-info*) y la formula como nuevas subconsultas.

Estas nuevas preguntas se concatenan a la consulta original del usuario, enriqueciendo el enfoque, y se reinicia el flujo completo del sistema agéntico (desde el enrutador hasta la recuperación y generación). Este ciclo de autocorrección puede repetirse hasta un máximo de tres iteraciones. De este modo, el sistema es capaz de autocorregir deficiencias en la primera recuperación y profundizar en la base de conocimiento de forma autónoma hasta lograr una respuesta precisa, trazable y completa.

4.2. Evaluación del Sistema

Para validar rigurosamente el rendimiento y la viabilidad de la arquitectura propuesta, se ha diseñado e implementado una metodología de evaluación cuantitativa. Dado que los sistemas RAG (y en especial los basados en grafos) presentan desafíos únicos en cuanto a la recuperación de conocimiento y la generación de lenguaje, es fundamental medir tanto la precisión del enrutamiento como la calidad de la respuesta final.

4.2.1. Diseño del *Benchmark*

Con este propósito, se ha construido un *benchmark* de validación propio compuesto por 50 preguntas cuidadosamente seleccionadas. Este conjunto de datos ha sido diseñado para estresar todas las ramificaciones del sistema agéntico, garantizando una cobertura exhaustiva de las distintas tipologías de consulta que el sistema debe ser capaz de resolver.

Las preguntas del *benchmark* se categorizan en los siguientes escenarios operativos (ilustrados con ejemplos del dominio zoológico):

- **Interacción y Alcance (*Greetings / Out of Scope*)**: Consultas de control de diálogo o preguntas ajenas al dominio (ej. "*What can you do?*" o "*¿Qué tiempo hace hoy?*").
- **Recuperación Directa y Mapeo de Entidades**: Búsquedas de propiedades exactas asociadas a un nodo (ej. "*¿Cuál es el nombre científico del león africano?*").

- **Agregación:** Consultas que requieren operaciones matemáticas o conteos sobre la base de datos (ej. “¿Qué familia taxonómica tiene el mayor número de especies?”).
- **Filtrado Estructurado:** Peticiones que imponen condiciones estrictas sobre las propiedades de los nodos (ej. “Lista todos los mamíferos que pesen más de 50 kg y vivan en la sabana”).
- **Datos Faltantes (*Missing Data*):** Preguntas sobre información que intencionadamente no está en el grafo, diseñadas para evaluar la robustez del sistema y la correcta activación de su respuesta de contingencia.
- **Consultas Semánticas:** Preguntas conceptuales y cualitativas que requieren comprensión del texto no estructurado (ej. “¿Cómo funciona el mecanismo de camuflaje del camaleón?”).
- **Razonamiento Complejo (*Multi-hop*):** Escenarios analíticos que requieren la invocación del agente planificador (ej. “Compara las estrategias de caza de los felinos africanos frente a los asiáticos”).

4.2.2. Métricas de Evaluación

Para obtener una visión holística del rendimiento del sistema en cada una de estas 50 iteraciones, se ha establecido la medición de cuatro métricas clave. La primera evalúa el comportamiento del agente de control, mientras que las tres restantes son métricas estándar en el estado del arte para la evaluación de arquitecturas RAG:

1. **Precisión de Selección de Herramienta (*Tool Selection Accuracy*):** Evalúa de forma binaria o porcentual si el Agente Enrutador (*Router*) ha seleccionado la herramienta o combinación de herramientas óptima para resolver la intención de la pregunta, validando así la lógica de decisión del sistema.
2. **Recuerdo del Contexto (*Context Recall*):** Mide la capacidad de los componentes de recuperación (*retrievers*) para extraer toda la información necesaria. Un alto *context recall* indica que el fragmento de texto o el subgrafo devuelto contiene los datos suficientes para responder a la pregunta, independientemente de cómo se redacte después.
3. **Fidelidad (*Faithfulness*):** Esta métrica cuantifica en qué medida la respuesta generada se deriva estrictamente del contexto recuperado. Actúa como el principal indicador de alucinaciones; una puntuación perfecta significa que el modelo no ha inventado ningún dato ni ha recurrido a su conocimiento paramétrico preentrenado.
4. **Corrección de la Respuesta (*Answer Correctness*):** Compara la respuesta generada por el sistema con una respuesta ideal de referencia (*ground truth*). Esta métrica evalúa la precisión semántica y factual de la salida, determinando si, en última instancia, el sistema ha resuelto de forma correcta y completa la necesidad de información del usuario.

El análisis conjunto de estas cuatro métricas permitirá identificar cuellos de botella en la arquitectura, discerniendo si los errores provienen de una mala decisión de enrutamiento, una recuperación deficiente o una alucinación en la fase de síntesis.